



Compiling Kotlin Coroutines to WebAssembly with Stack-Switching

Veronika Sirotkina
Aleksandr Shefer

Context: Kotlin Coroutines

- Currently Kotlin Coroutines in Wasm are implemented via state machines.

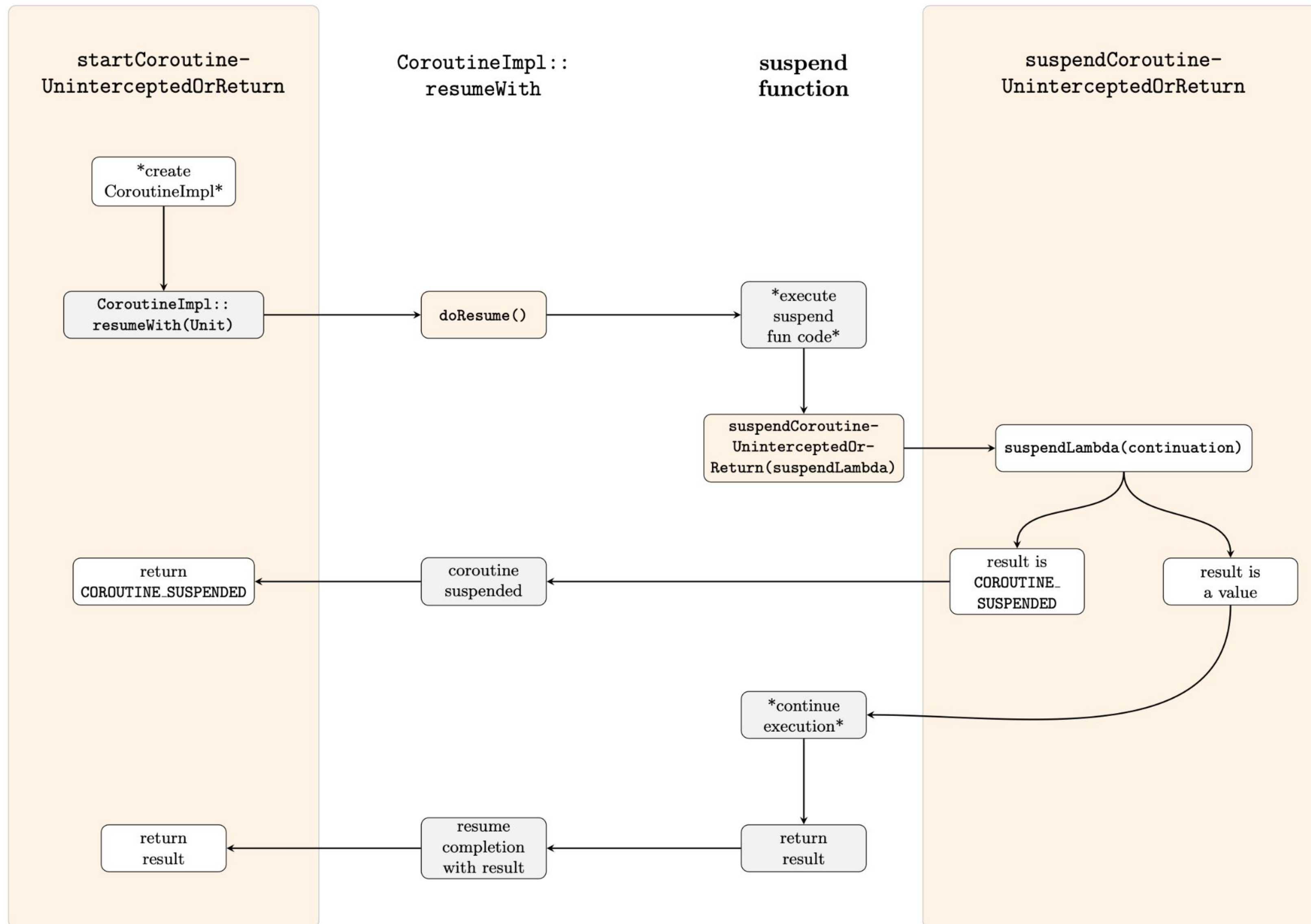
Before state-machine transformation

```
val a: suspend () → Unit = {  
    println(1 + suspendMe())  
}
```

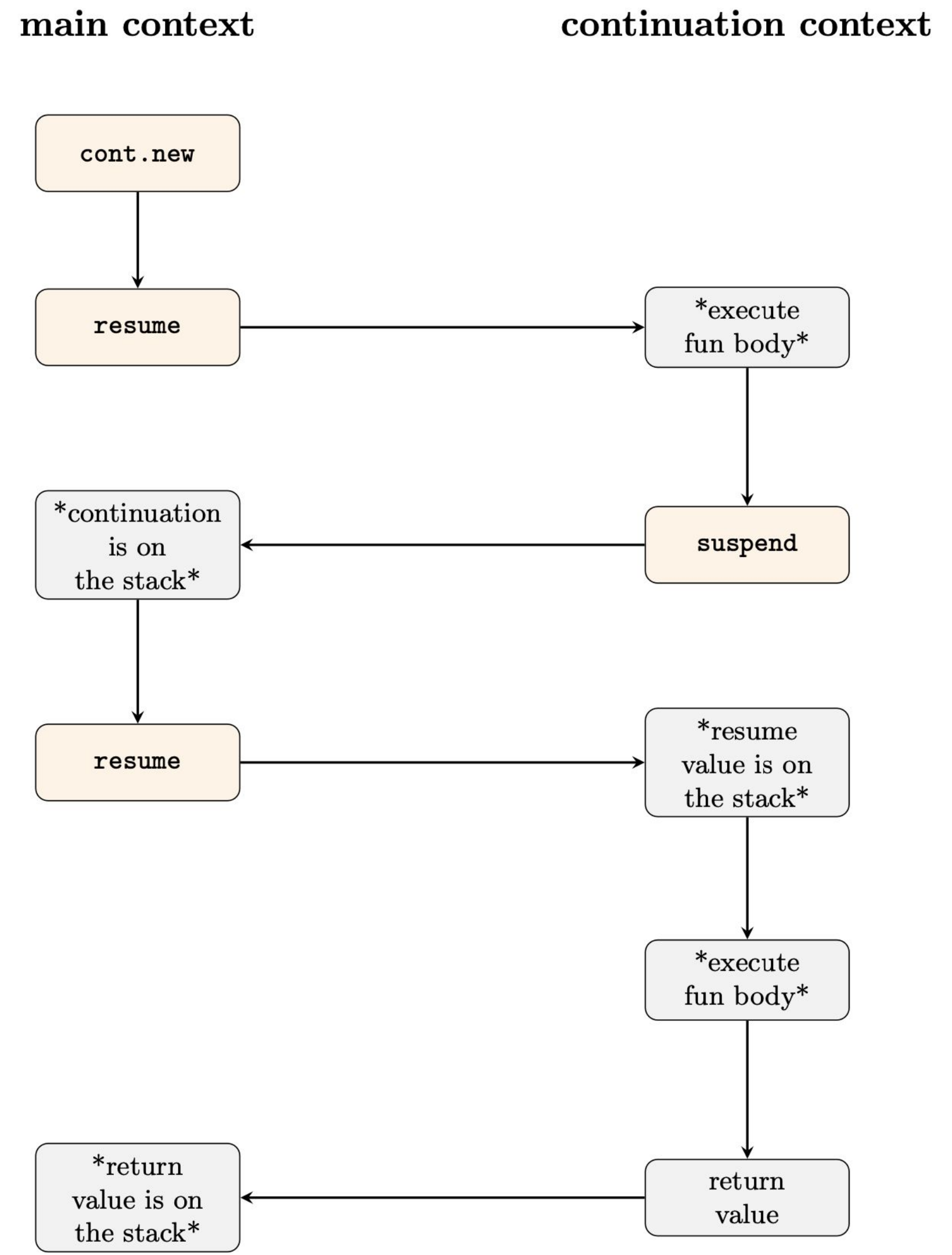
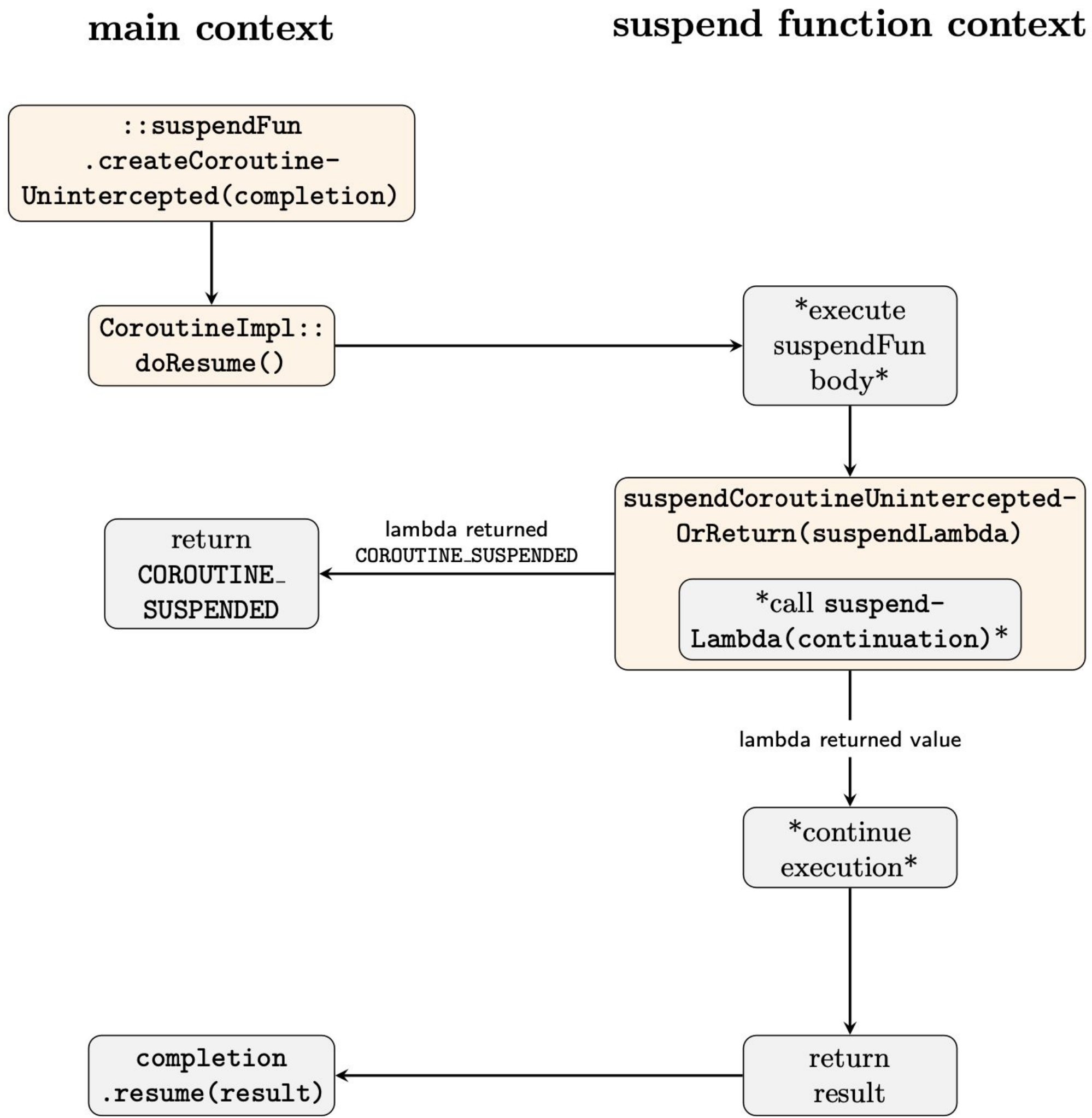
After state-machine transformation

```
fun invokeSuspend($result: Any?): Any? {  
    when (this.label) {  
        0 → {  
            this.label = 1  
            $result = suspendMe(this)  
            if ($result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED  
            goto 1  
        }  
        1 → {  
            $result.throwOnFailure()  
            println(1 + $result)  
            return Unit  
        }  
        else → {  
            throw IllegalStateException()  
        }  
    }  
}
```

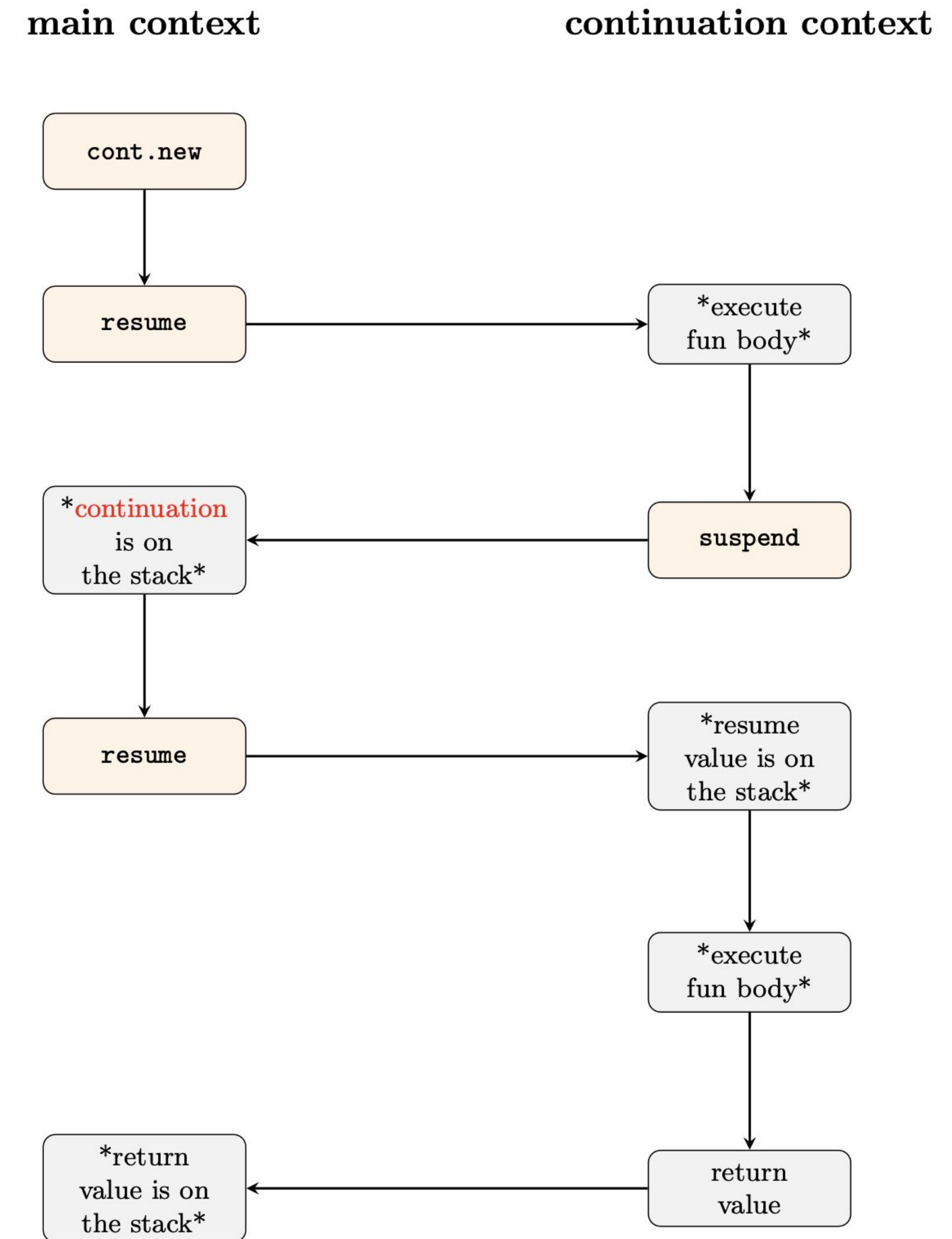
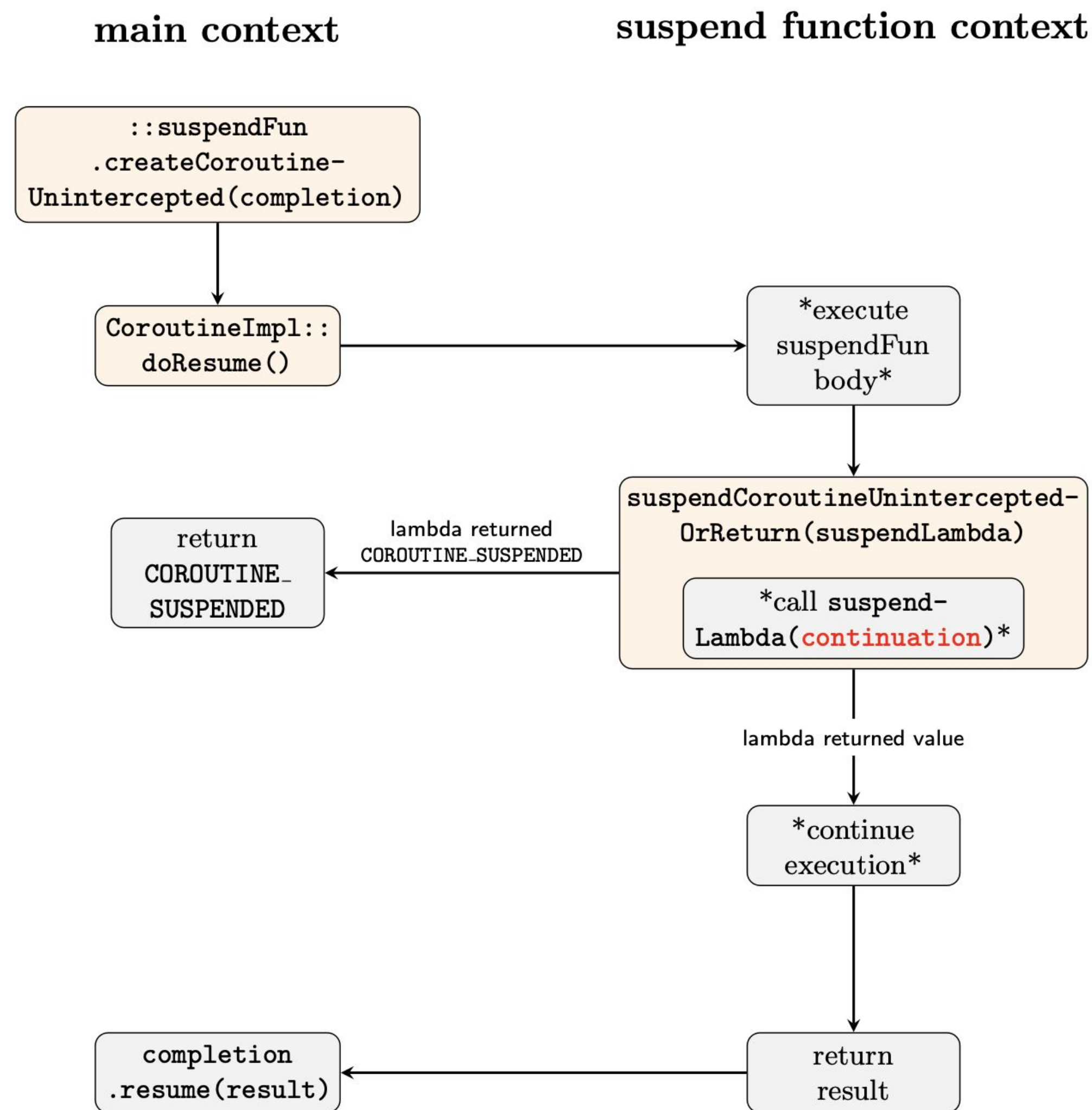
Kotlin Coroutines



Coroutines vs Stack-Switching primitives



Continuation object availability

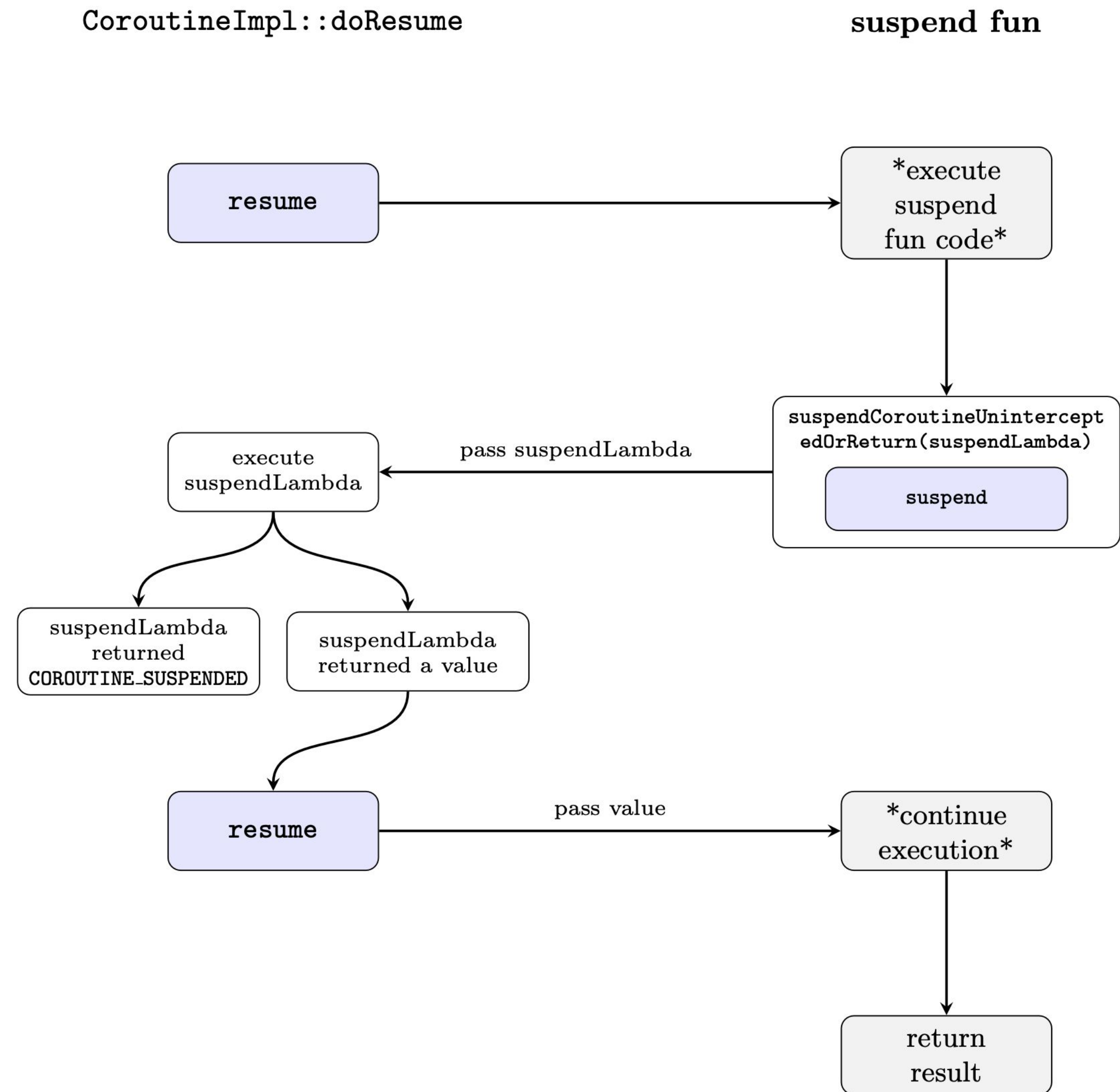


Immediate resume problem

```
suspendCoroutineUninterceptedOrReturn { continuation →  
    continuation.resume("OK")  
    COROUTINE_SUSPENDED  
}
```

Initial solution

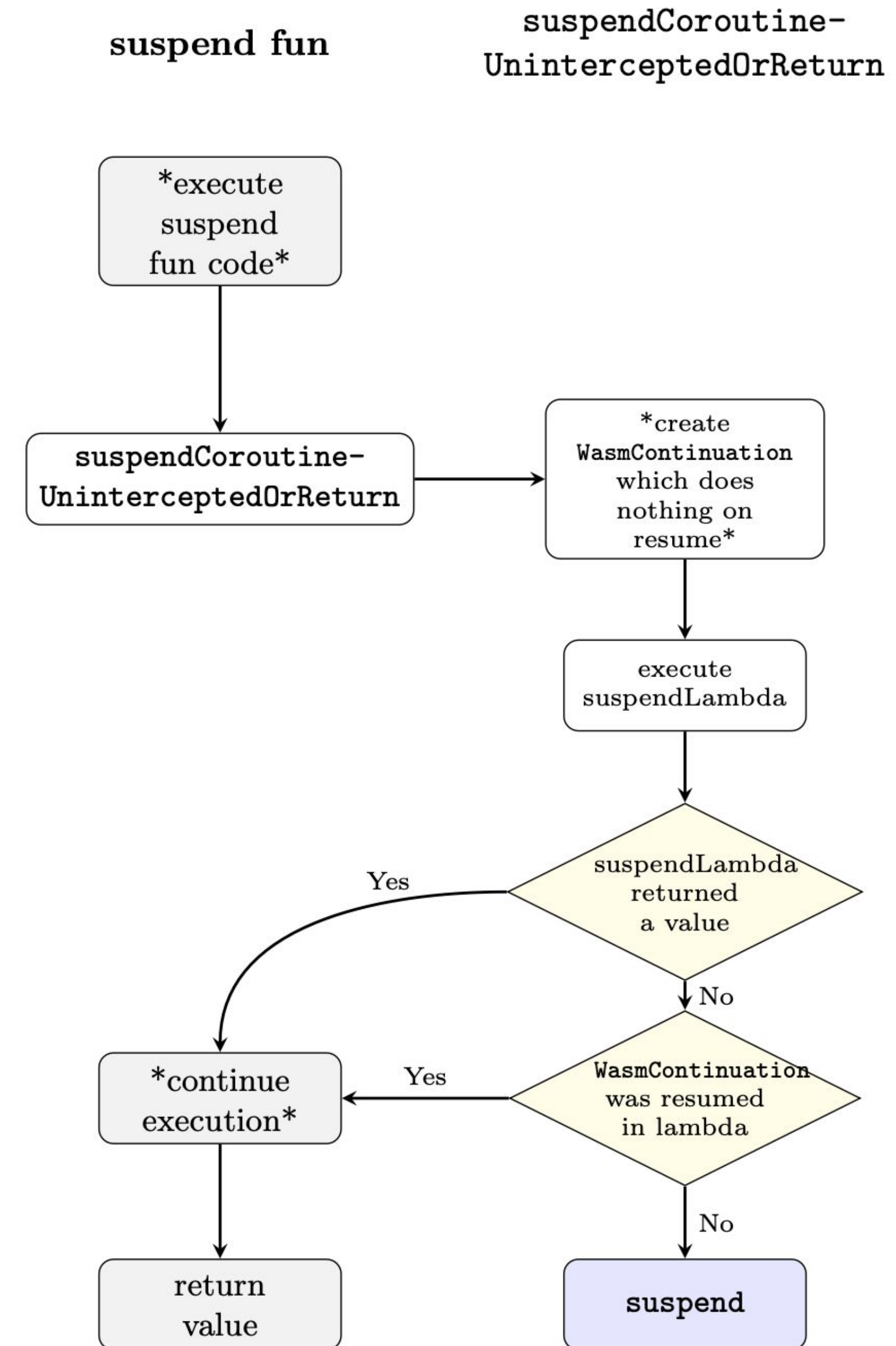
- suspend unconditionally
- pass suspend lambda to the call site via suspend instruction
- execute on call site
- resume with the block's result



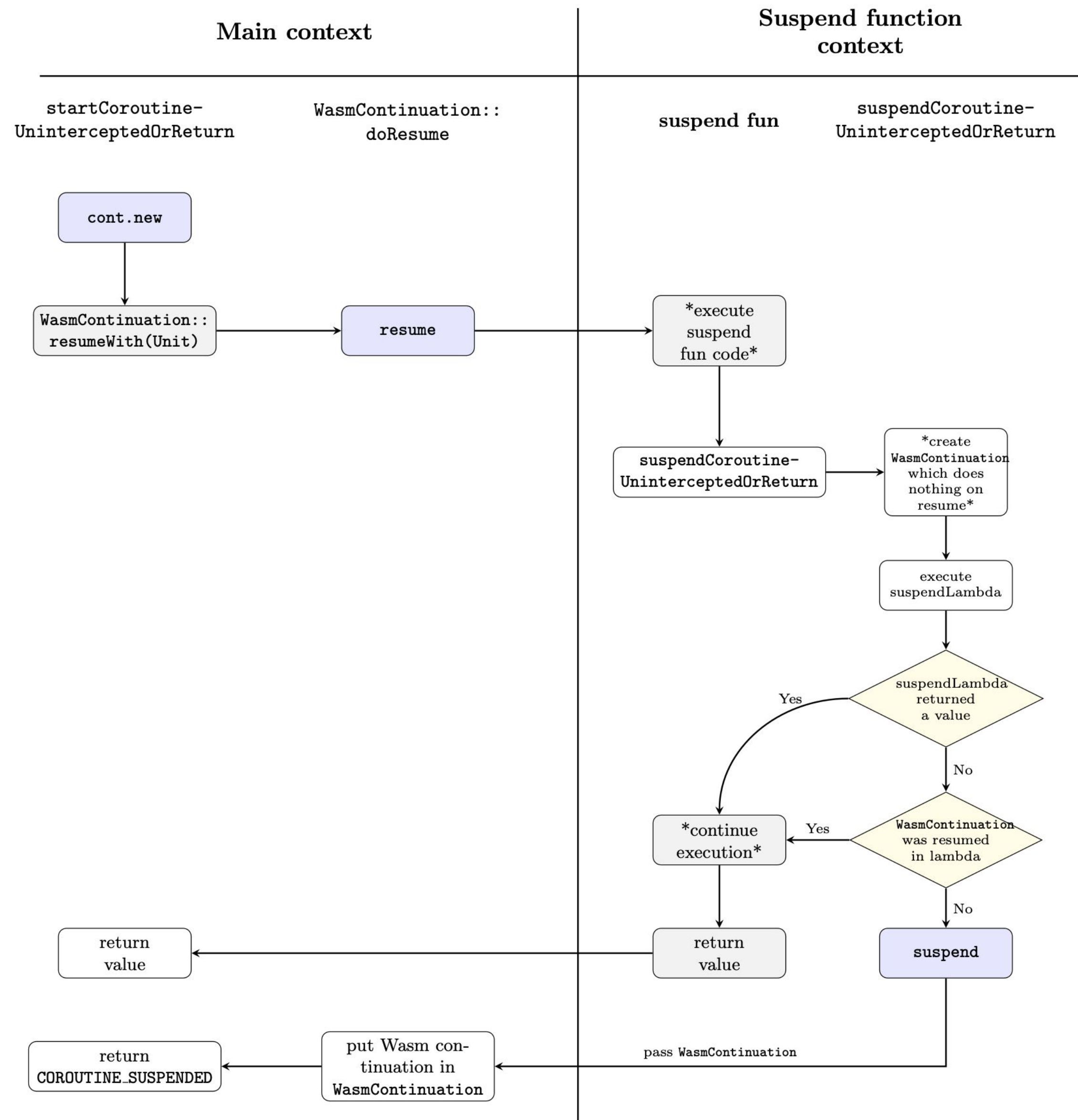
Optimization

Actually, resuming continuation inside suspend is the same as not suspending at all.

Let's... not do anything! Almost...



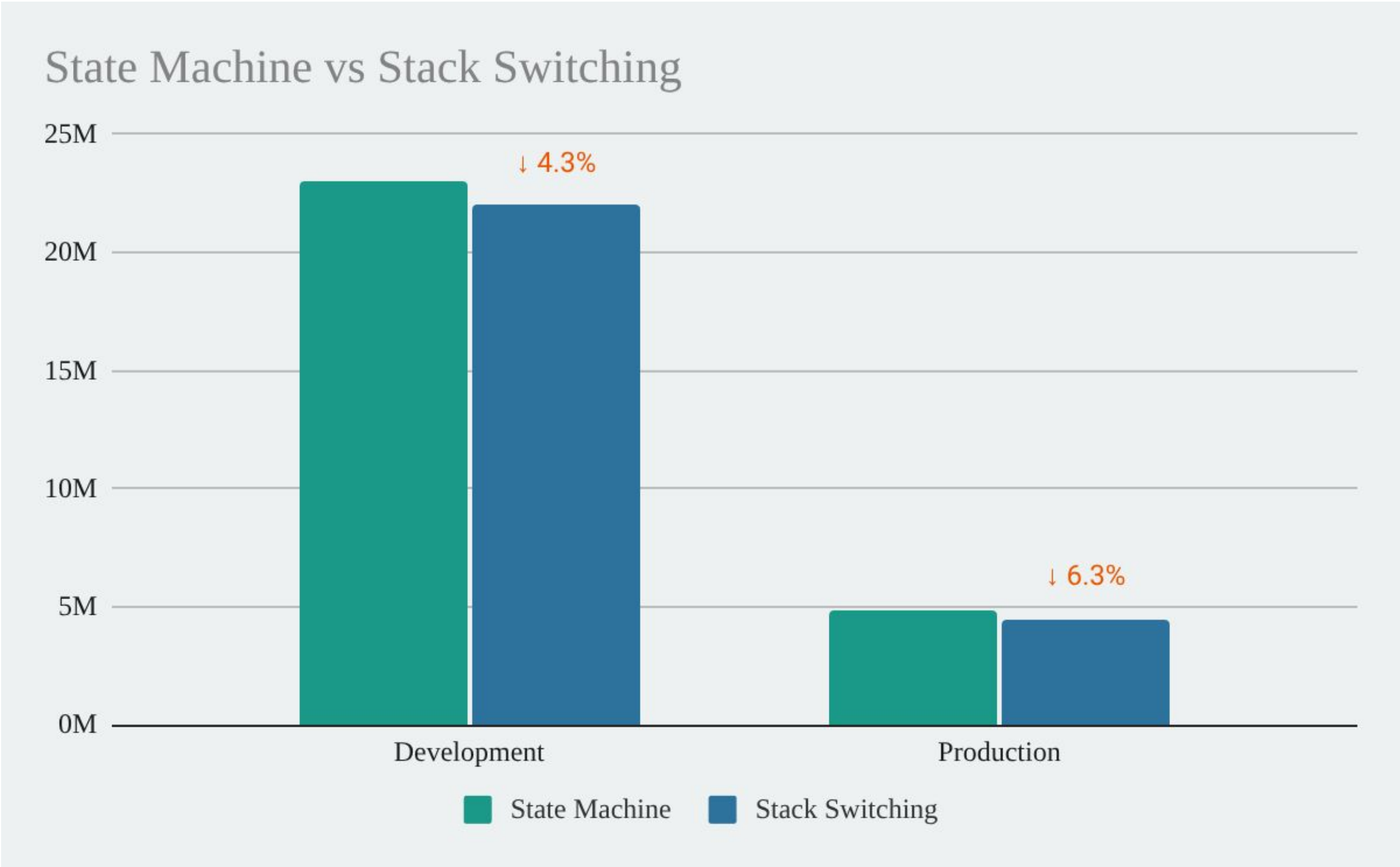
Final implementation



Benchmarks

(V8)

Size Benchmarks (KotlinConf App)



	State Machine	Stack Switching	Decrease
Development	23M	22M	-4.3%
Production	4.8M	4.5M	-6.3%

Coroutine performance on kotlin-wasm-benchmarks

	State Machine (ms)	Stack Switching (ms)	Ratio
coroutineIteration	19.08	45.36	2.377
coroutineRecursion	183.95	312.78	1.700
takeWhileDirect	4.82	2.44	0.505
cancellableReusable	14.04	600.36	42.769
externalResumptions	0.15	2.91	18.902
createCoroutine	0.10	33.07	348.084
startCoroutine	0.12	2.33	33.508

<https://github.com/Kotlin/kotlin-wasm-benchmarks>

Coroutine performance on Compose benchmarks

Benchmark	State Machine	Stack Switching	Unit	Ratio
AnimatedVisibility	156.68	96.73	us	0.62
CanvasDrawing	8.39	8.48	ms	1.01
HeavyShader	1.96	2.07	ms	1.06
LazyGrid	6.87	6.77	ms	0.99
LazyList	194.02	199.74	us	1.03
MultipleComponents-NoVectorGraphics	395.38	299.45	us	0.76

<https://github.com/JetBrains/compose-multiplatform/tree/master/benchmarks/multiplatform/benchmarks>

Current state

- Binary sizes are slightly smaller (5-10%)
- Stack-Switching-based Coroutine implementation is slower than State-Machine-based
 - For pure Kotlin coroutine intrinsics (suspend+resume) benchmark results can be 30x slower
 - Examples closer to real-world user applications show lower performance gap

Appendix: Some Benchmarks (code, CoroutineIteration/CoroutineRecursion)

* Benchmark pipeline *

1. Generator & Caller coroutines

yieldValue: generator suspends and resumes caller (with the current result)

nextValue: caller suspends and resumes generator

2. Generator initialization

suspendCoroutine { continuation →

 generatorContinuation = continuation

}

generatorBody() // contains computations and calls of yieldValue

generatorContinuation = null

3. Caller: iteratively fetching values from generator

while (generator.hasNext()) {

 sum += generator.nextValue()

}

Appendix: Some Benchmarks (code)

`* CoroutineRecursion benchmark *`

```
suspend fun generatorBody() = runBibonacci(n);
```

```
suspend fun runFibonacci(a: Int): Int {  
    return when (a) {  
        0, 1 → a.also { yieldValue(it) }  
        else →  
            (runFibonacci(a - 1) + runFibonacci(a - 2)).also {  
                yieldValue(it)  
            }  
    }  
}
```

`* CoroutineIteration benchmark *`

```
suspend fun generatorBody() {  
    for (i in rangeStart..rangeEnd step 1) {  
        yieldValue(i)  
    }  
}
```


Appendix: Internals

```
private var generatorContinuation: Continuation<Unit>? = null
private var callerContinuation: Continuation<T>? = null
```

```
protected suspend fun yieldValue(x: T) {
    suspendCoroutine { continuation →
        generatorContinuation = continuation
        callerContinuation?.resume(x)
    }
}
```

```
suspend fun nextValue(): T {
    return suspendCoroutine { continuation →
        callerContinuation = continuation
        generatorContinuation?.resume(Unit)
    }
}
```

Appendix: Internals

```
internal suspend inline fun <T> suspendCoroutineUninterceptedOrReturn(block: (Continuation<T>) → Any?): T {
    val completion = getContinuation<T>()
    val wasmContBox = WasmContinuationBox(null) // by now we don't have `contref`
    val freshCont = CoroutineImpl<T, T>(completion, wasmContBox)

    val blockResult = block(freshCont)

    if (blockResult == COROUTINE_SUSPENDED) return blockResult as T

    suspendIntrinsic(wasmCont.wasmContBox) // wasm `suspend` passing continuation box

    val e = freshCont.exception
    if (e  $\neq$  null) throw e
    return freshCont.result as T
}
```

Appendix: Internals

```
override fun doResume(): Any? {  
    val wasmCont = wasmContBox.wasmContinuation!!  
  
    val resumeResult: Any? = exception?.let {  
        resumeThrowImpl(it, wasmCont)  
    } ?: resumeWithImpl(wasmCont)  
  
    return resumeResult  
}
```

```
func $resumeWithImpl (type ...)  
    (param $wasmCont (ref null $cont)) (result (ref null $Any))  
    block (result (ref null $Any) (ref null $cont))  
        local.get $wasmCont  
        resume $cont 1 (on 1 0) // tag 1 for suspend/resume  
        return  
    end  
    // currently wasm Continuation on stack - save it to WasmContinuationBox  
    local.set $wasmCont  
    ref.cast $WasmContinuationBox  
    local.get $wasmCont  
    struct.set (type $WasmContinuationBox) 4  
    call $<get-COROUTINE_SUSPENDED>  
    return
```